

Reasoning Pipeline: proposal_planning → CodeAgent → Final Answer

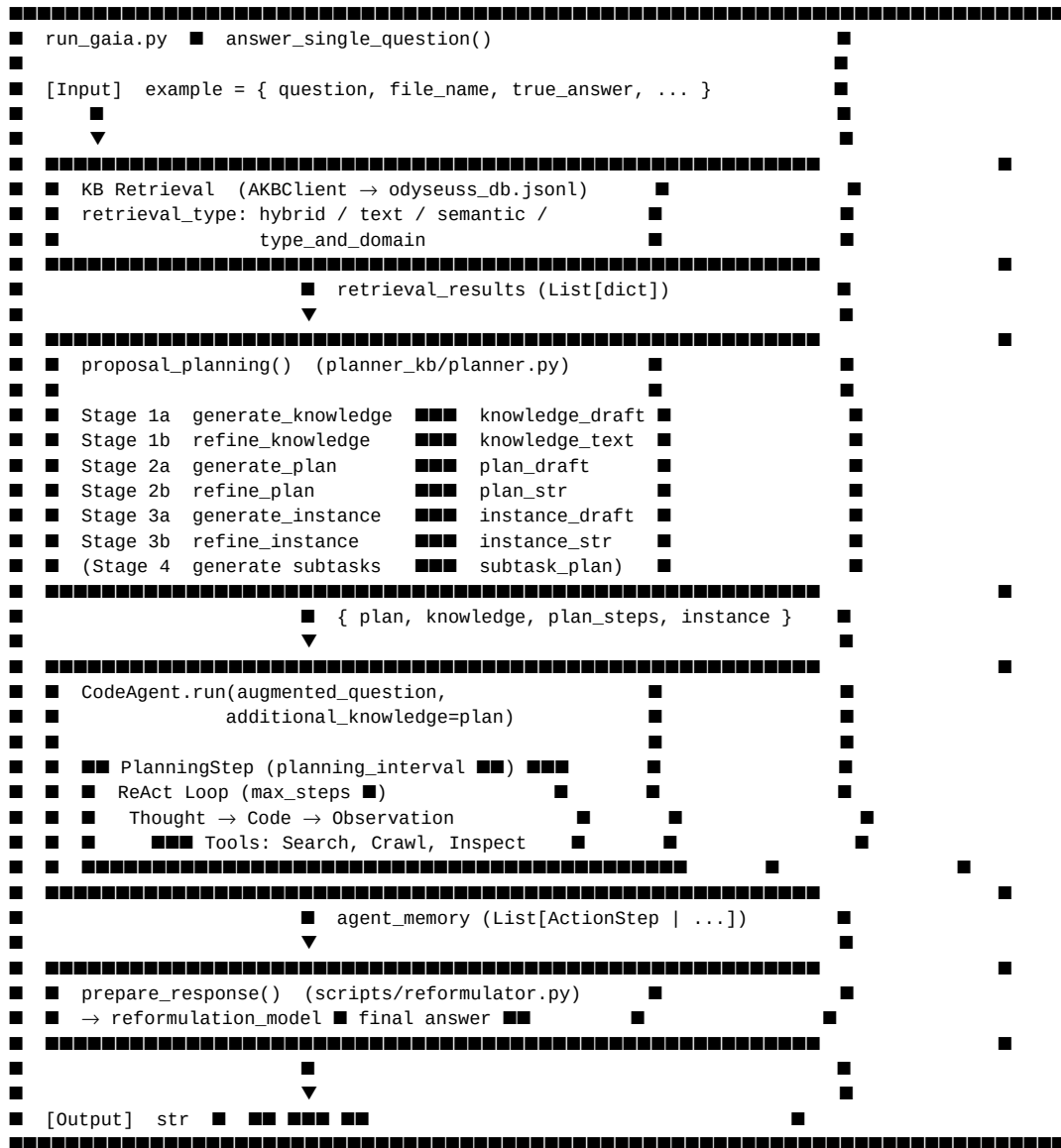
> 작성일: 2026-04-25

> 대상 코드베이스: examples/open_deep_research/

> 언어: Python 3 / smolagents / sentence-transformers

1. 전체 흐름 개요 (Overview)

아래 ASCII 흐름도는 GAIA 벤치마크 태스크 하나를 처리하는 전체 파이프라인을 보여줍니다.



핵심 모듈 간 의존 관계:

모듈	역할
`run_gaia.py`	전체 오케스트레이터. `answer_single_question()` 진입점
`agent_kb/agent_kb_retrieval.py`	`AgenticKnowledgeBase` + `AKB_Manager` — KB 인덱싱 및 검색

`planner_kb/planner.py`	`proposal_planning()` — 7단계 지식·계획·인스턴스 생성
`planner_kb/planner_prompts.yaml`	모든 프롬프트 템플릿
`smolagents.CodeAgent`	ReAct 루프 실행
`scripts/reformulator.py`	에이전트 메모리 → 최종 답변 추출

2. KB 조회 단계 (KB Retrieval)

2.1 데이터 소스 — `odysseuss_db.jsonl`

각 레코드는 하나의 과거 태스크 경험을 표현하며 다음 최상위 필드를 가집니다.

필드	타입	설명	
`task_id`	`str`	레코드 고유 ID	
`task`	`str`	태스크 질문 텍스트 (또는 `question` 키)	
`true_answer`	`str`	정답 레이블	
`task_analysis`	`dict`	{ `knowledge`, `plan`, `task_type`, `domain` }	
`agent_planning`	`str` or `null`	null	에이전트가 생성한 원시 계획 텍스트
`decision_augmentation`	`dict`	{ `final_reference`: { `knowledge`, `plan`, `instance` }, `signals_summary`: [...] }	
`plan_subtask_action`	`list` or `null`	null	{ { `step`, `subtasks`: [{ `subtask`, `subtask_action` }] } }

`parse_json_file()` 함수는 파일 로딩 시 다음 정규화를 수행합니다.

```
# task_analysis.task_type / domain : dict → list ■■
if isinstance(ta_type, dict):
    task_analysis["task_type"] = [ta_type.get("raw"), ta_type.get("normalized")]

# decision_augmentation.final_reference ■ knowledge/plan ■ task_analysis ■■■
if not task_analysis.get("knowledge") and fr.get("knowledge"):
    task_analysis["knowledge"] = fr["knowledge"]
```

2.2 검색 모드 (Retrieval Modes)

`AKB_Manager` 클래스(`agent_kb_retrieval.py`)는 네 가지 검색 API를 제공합니다.

`hybrid_search(query, top_k, weights)`

TF-IDF 텍스트 점수와 sentence-transformer 의미 유사도를 가중 합산합니다.

```
score = weights["text"] × TF-IDF cosine similarity
+ weights["semantic"] × all-MiniLM-L6-v2 embedding cosine similarity
```

기본 가중치: { "text": 0.5, "semantic": 0.5 }

`search_by_text(query, field, top_k)`

TF-IDF 벡터라이저로 `task` 필드에 대한 코사인 유사도 검색. `sklearn.TfidfVectorizer(stop_words="english")` 사용.

`search_by_semantic(query, field, top_k)`

`sentence-transformers/all-MiniLM-L6-v2` 모델로 임베딩한 뒤 코사인 유사도로 순위 결정.

`type_domain_text_search(query, task_types, domains, top_k, weights)`

type-domain 오버랩 점수와 TF-IDF 점수를 결합한 가중 합산 검색:

```
final_score = 0.3 × type_score + 0.3 × domain_score + 0.4 × text_score
```

```
type_score = |query_types ∩ kb_types| / max(|query_types|, 1)
```

```
domain_score = |query_domains ∩ kb_domains| / max(|query_domains|, 1)
```

2.3 검색 결과 딕셔너리 스키마

`AKB_Manager.hybrid_search()` / `type_domain_text_search()` 의 반환 형식:

```
{
  "task_id": str,
  "total_score": float, # hybrid: TF-IDF + semantic ■■
  # (type_domain: "score" + "overlap_count")
  "task": str,
  "true_answer": str,
  "agent_planning": str | None,
  "agent_experience": str | None,
  "task_analysis": {
    "knowledge": str,
    "plan": list[str],
    "task_type": list[str],
    "domain": list[str],
  },
  "plan_subtask_action": list | None,
  "instance": str | None, # decision_augmentation.final_reference.instance
  "decision_guide": list | None, # decision_augmentation.signals_summary
}
```

`decision_guide` 항목 하나의 형식:

```
{
  "level": "knowledge" | "plan" | "instance",
  "failures": list[str], # ■■ ■■ "failure": str
  "causes": list[str], # ■■ ■■ "cause": str
  "corrections": list[str], # ■■ ■■ "correction": str
}
```

3. `proposal_planning` 함수 (Proposal Planning)

3.0 함수 시그니처

```
def proposal_planning(
    example, # GAIA ■■■ dict (question, file_name, ...)
    augmented_question: str, # ■■ ■■■ ■■ ■■■ ■■ ■■■
    model_name: str,
    key: str,
    url: str,
    model: Any,
    slm: bool,
    retrieval_method: Callable, # AKBClient.hybrid_search ■
    top_k: int,
```

```

retrieval_option: Literal["task_text", "type_and_domain"] = "task_text",
type_domain_retrieval_method=None,
plan_mode: Optional[str] = None, # "plan" | "subtask" | "plan_subtask" | ...
tools=None,
managed_agents=None,
planning_prompt_templates=None,
) -> dict

```

반환값:

```

{
    "plan": str, # CodeAgent ■■■■ ■■■■■■
    "knowledge": str, # refined knowledge text
    "plan_steps": str, # refined plan (bullet list)
    "instance": str, # refined instance
    "retrieval_results": list,
    "examples": dict, # subtask_examples dict
}

```

3.1 단계 [1] — 유사 태스크 검색 (Retrieve Similar Tasks)

목적: 현재 질문과 가장 유사한 KB 레코드 **top_k**개를 가져와 이후 단계의 few-shot 예시와 Decision Guide로 활용.

입력:

- **example["question"]** — 원본 질문 텍스트
- **retrieval_option** — "task_text" 또는 "type_and_domain"

처리 흐름:

```

if retrieval_option == "type_and_domain":
    # 1) LLM■■■ ■■■■■■ ■■■■
    classification = classify_task_type_and_domain(task=augmented_question, ...)
    task_types = classification["task_type_normalized"]
    domains = classification["domain_normalized"]
    # 2) type+domain+text ■■■■■■ ■■■■
    retrieval_results = type_domain_retrieval_method(
        example["question"], task_types=task_types, domains=domains, top_k=top_k
    )
    # fallback: ■■■■■■ text ■■■■■■ ■■■■
else:
    retrieval_results = retrieval_method(example["question"], top_k=top_k)

```

검색 후 레벨별 블록 구성:

```

knowledge_examples = build_similar_task_direction_blocks(retrieval_results)
plan_examples = build_similar_task_plan_blocks(retrieval_results)
instance_examples = build_instance_examples(retrieval_results)
guide_knowledge = build_decision_guide_blocks(retrieval_results, level="knowledge")
guide_plan = build_decision_guide_blocks(retrieval_results, level="plan")
guide_instance = build_decision_guide_blocks(retrieval_results, level="instance")

```

출력: **retrieval_results** (List[dict]) + 6개의 예시·가이드 문자열

3.2 단계 [2] — Stage 1a: generate_knowledge

목적: 주어진 태스크를 풀기 위해 필요한 도메인 지식(선언적 + 절차적)을 생성.

함수 시그니처:

```

def generate_knowledge(
    task: str,
    examples: str, # knowledge_examples ■■■
    planning_prompt_template: dict,
    model_name: str, key: str, url: str, model: Any, slm: bool,

```

```
) -> str
```

프롬프트 템플릿 (`planner_prompts.yaml` — `knowledge_prompt`):

```
knowledge_prompt: |
  Extract the domain knowledge required to understand and solve the given task.
  Provide both:
    (a) declarative knowledge: domain concepts and definitions used
    (b) procedural knowledge: methodology, paradigm, or algorithm to apply
  Return only the knowledge as plain text. Do NOT include the direct answer.

  TASK:
  {{task}}

  SIMILAR TASKS:
  {{examples}}
```

LLM 호출: `call_model(query=prompt, model_name, key, url, model, slm)`

출력: `knowledge_draft` (str) — 선언적·절차적 지식 텍스트

3.3 단계 [3] — Stage 1b: `refine_knowledge` (with Decision Guide)

목적: Stage 1a의 초안 지식을 Decision Guide 신호로 교정. `decision_guide`가 비어 있으면 draft를 그대로 반환.

함수 시그니처:

```
def refine_knowledge(
    task: str,
    draft: str,          # generate_knowledge ■■■
    decision_guide: str, # guide_knowledge ■■
    planning_prompt_template: dict,
    model_name: str, key: str, url: str, model: Any, slm: bool,
) -> str
```

프롬프트 템플릿 (`planner_prompts.yaml` — `refine_knowledge_prompt`):

```
refine_knowledge_prompt: |
  You generated domain knowledge for a task. A Decision Guide identifies failure patterns
  observed in similar tasks.
  Refine the generated knowledge to address the failures. Only modify what the guide says
  is insufficient or wrong.
  Preserve all correct parts. Return the full refined knowledge as plain text.

  TASK:
  {{task}}

  GENERATED KNOWLEDGE:
  {{draft}}

  DECISION GUIDE:
  {{decision_guide}}
```

출력: `knowledge_text` (str) — 교정된 최종 지식

3.4 단계 [4] — Stage 2a: `generate_plan`

목적: 지식과 유사 태스크 예시를 바탕으로 최대 10단계 고수준 계획을 생성.

함수 시그니처:

```
def generate_plan(
    task: str,
    knowledge: str,          # knowledge_text
    examples: str,          # plan_examples ■■
)
```

```

    planning_prompt_template: dict,
    model_name: str, key: str, url: str, model: Any, slm: bool,
) -> str

```

프롬프트 템플릿 (`planner_prompts.yaml` — `plan_prompt`):

```

plan_prompt: |
    Develop a high-level plan of no more than 10 steps to solve the given task based only
    on the provided knowledge and similar tasks.
    The plan must not include the final answer and the final conclusion.

    Requirements:
    - Use the knowledge explicitly and progressively in each step.
    - Derive a decision criterion: an intermediate, task-specific quantity, rule, or structure
      that can be used to select the answer.

    Strict Output Format:
    - <step>
    - <step>
    - ...

    TASK:
    {{task}}

    KNOWLEDGE:
    {{knowledge}}

    SIMILAR TASKS:
    {{examples}}

```

출력: `plan_draft` (str) — bullet-list 형식의 단계별 계획

3.5 단계 [5] — Stage 2b: `refine_plan` (with Decision Guide)

목적: Stage 2a의 계획 초안을 Decision Guide 신호로 교정.

함수 시그니처:

```

def refine_plan(
    task: str,
    knowledge: str,
    draft: str,          # plan_draft
    decision_guide: str, # guide_plan ■■
    planning_prompt_template: dict,
    model_name: str, key: str, url: str, model: Any, slm: bool,
) -> str

```

프롬프트 템플릿 (`planner_prompts.yaml` — `refine_plan_prompt`):

```

refine_plan_prompt: |
    You generated a solution plan for a task. A Decision Guide identifies failure patterns
    observed in similar tasks.
    Refine the generated plan to address the failures. Only modify steps that the guide says
    are wrong or missing.
    Preserve all correct steps. Return the full refined plan as a bullet list.

    TASK:
    {{task}}

    KNOWLEDGE:
    {{knowledge}}

    GENERATED PLAN:
    {{draft}}

    DECISION GUIDE:
    {{decision_guide}}

```

출력: `plan_str` (str) — 교정된 최종 계획

3.6 단계 [6] — Stage 3a: `generate_instance`

목적: 계획을 실제 태스크에 구체적으로 적용한 실행 인스턴스를 생성. 중간 계산값과 추론 과정을 포함하되 최종 답은 제외.

함수 시그니처:

```
def generate_instance(
    task: str,
    knowledge: str,
    plan: str,                # plan_str
    examples: str,           # instance_examples ■■■
    planning_prompt_template: dict,
    model_name: str, key: str, url: str, model: Any, slm: bool,
) -> str
```

프롬프트 템플릿 (`planner_prompts.yaml` — `instance_prompt`):

```
instance_prompt: |
    Generate a concrete execution instance that grounds the given plan into specific,
    actionable steps with actual values and reasoning.
    The instance should demonstrate exactly how to apply the plan to THIS task, showing
    intermediate computations and logical deductions.

    Requirements:
    - Walk through each plan step with concrete values from the task.
    - Show actual computations, lookups, or reasoning steps.
    - Do NOT reveal or guess the final answer – stop just before the conclusion.

    TASK:
    {{task}}

    KNOWLEDGE:
    {{knowledge}}

    PLAN:
    {{plan}}

    SIMILAR INSTANCES:
    {{examples}}
```

출력: `instance_draft` (str) — 구체적 실행 인스턴스 텍스트

3.7 단계 [7] — Stage 3b: `refine_instance` (with Decision Guide)

목적: Stage 3a의 인스턴스 초안을 Decision Guide로 교정.

함수 시그니처:

```
def refine_instance(
    task: str,
    knowledge: str,
    plan: str,
    draft: str,                # instance_draft
    decision_guide: str,       # guide_instance ■■■
    planning_prompt_template: dict,
    model_name: str, key: str, url: str, model: Any, slm: bool,
) -> str
```

프롬프트 템플릿 (`planner_prompts.yaml` — `refine_instance_prompt`):

```
refine_instance_prompt: |
    You generated a concrete execution instance for a task. A Decision Guide identifies
```

failure patterns observed in similar tasks.
Refine the instance to fix the failures identified in the guide. Only modify what is incorrect or incomplete.
Preserve all correct parts. Return the full refined execution instance.

```
TASK:
{{task}}

KNOWLEDGE:
{{knowledge}}

PLAN:
{{plan}}

GENERATED INSTANCE:
{{draft}}

DECISION GUIDE:
{{decision_guide}}
```

출력: `instance_str` (str) — 교정된 최종 인스턴스

3.8 단계 [선택] — Stage 4: Subtask 생성

`plan_mode`가 "subtask", "plan_subtask", "plan_subtask_action" 중 하나이고 `planning_prompt_templates`가 제공된 경우 활성화됩니다.

```
if plan_mode in ("subtask", "plan_subtask", "plan_subtask_action"):
    stage2_prompt = populate_template(
        planning_prompt_templates["initial_plan_subtask_action_stage2"],
        variables={
            "task": augmented_question,
            "tools": tools,
            "managed_agents": managed_agents,
            "plan_steps": plan_str,
            "examples": subtask_examples[example_key],
        },
    )
    subtask_plan = call_model(query=stage2_prompt, ...)
```

최종 `plan` 문자열 조합:

```
# plan_mode ( )
final_plan = f"Knowledge:\n{knowledge_text}\n\nPlan:\n{plan_str}\n\nInstance:\n{instance_str}"

# plan_mode (subtask )
final_plan = f"Knowledge:\n{knowledge_text}\n\nPlan:\n{plan_str}\n\nInstance:\n{instance_str}\n\nSubtasks:\n{subtask_plan}"
```

4. CodeAgent 실행 (CodeAgent Execution)

4.1 에이전트 생성

`create_agent_hierarchy()` 함수에서 `CodeAgent`를 초기화합니다.

```
manager_agent = CodeAgent(
    model=model,
    tools=[],
    max_steps=args.max_steps,          # 12
    verbosity_level=2,
    additional_authorized_imports=AUTHORIZED_IMPORTS,
    planning_interval=args.planning_interval, # 1
    managed_agents=[],
    debug=debug,
```


도구	클래스	역할
`SearchTool`	`scripts/searcher.py`	Tavily / Exa 등 웹 검색
`CrawlerReadTool`	`scripts/async_web_crawler.py`	URL 본문 크롤링
`CrawlerArchiveSearchTool`	`scripts/async_web_crawler.py`	Wayback Machine 아카이브 검색
`TextInspectorTool`	`scripts/text_inspector_tool.py`	PDF/텍스트 파일 내용 검사
`AudioInspectorTool`	`scripts/audio_inspector_tool.py`	오디오 파일 내용 검사
`VisualInspectorTool`	`scripts/visual_inspector_tool.py`	이미지 파일 내용 검사

도구들은 `create_agent_hierarchy()` 호출 전에 인스턴스화되고, `CodeAgent.tools` 리스트에 등록됩니다.

4.5 메모리 객체

에이전트 메모리는 세 가지 `smolagents.memory` 타입으로 구성됩니다.

타입	용도
`TaskStep`	에이전트에 최초로 부여된 태스크와 `additional_knowledge`
`PlanningStep`	`planning_interval`마다 LLM이 작성하는 고수준 계획
`ActionStep`	각 반복의 (Thought, Code, Observation) 삼중 쌍

실행 완료 후 `agent.write_memory_to_messages(summary_mode=True)`로 전체 메모리를 메시지 리스트로 직렬화합니다.

5. 최종 답변 추출 (Final Answer Extraction)

5.1 `prepare_response()` — `scripts/reformulator.py`

```
def prepare_response(
    original_task: str,
    inner_messages,          # agent.write_memory_to_messages() ■■■
    reformulation_model: Model,
    multiple: bool = False,
) -> str
```

처리 과정:

1. 시스템 메시지로 원래 태스크를 제시
2. 에이전트 메모리(`inner_messages`)를 USER 역할로 변환하여 대화 이력 구성
3. 최종 지시 메시지 추가 — 단답형 포맷 규칙 명시

최종 지시 메시지 원문:

Read the above conversation and output a FINAL ANSWER to the question. The question is repeated here for convenience:

{original_task}

FINAL ANSWER FORMAT: Your response must strictly follow these formatting rules:

- For NUMBERS: Use digits only (not words), omit commas and units (no \$, USD, %, etc.) unless specifically requested
- For TEXT: Omit articles and abbreviations unless specified, exclude final punctuation (!?)
- For LISTS: Provide comma-separated values following the above number/text rules
- Follow ALL formatting instructions in the original question (alphabetization, sequencing, decimal places, etc.)
- Please carefully understand the requirements of the original task and ensure that the

- final output meets the specific units given in the question (/Angstrom, /thousand hours, etc.)
- If you cannot determine an answer, respond only with: "Unable to determine"
- Your entire response should consist of ONLY the requested information in the EXACT format specified - nothing more, nothing less.

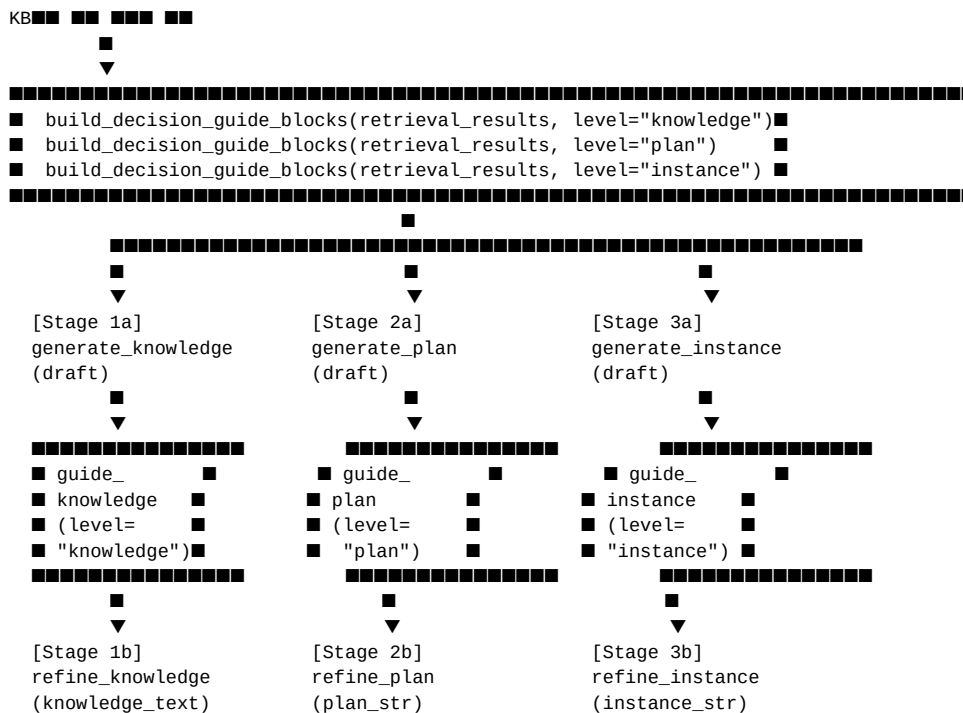
4. `reformulation_model(messages).content` 호출

5. `response.split("FINAL ANSWER: ")[-1].strip()` 로 최종 답 추출

출력: `str` — 숫자·텍스트·리스트 형식이 정제된 단답형 최종 답변

6. Decision Guide 교정 흐름 (Decision Guide Refine Flow)

6.1 교정 흐름도



6.2 Decision Guide 신호 구조

```

# decision_guide
{
  "level": "knowledge" | "plan" | "instance",
  "failures": ["1", "2", ...],
  "causes": ["1", "2", ...],
  "corrections": ["1", "2", ...],
}

```

`build_decision_guide_blocks()` 함수가 이를 프롬프트 삽입용 텍스트로 변환합니다:

```

[From Task: <task text>]
[KNOWLEDGE] Failure: <failures[0]>
Cause: <causes[0]>
Correction: <corrections[0]>
[PLAN] Failure: <failures[1]>
...

```

6.3 Generate-then-Refine 전략의 장점 (SLM 관점)

전략	특성
단일 패스 생성	SLM은 긴 Guide 조건을 프롬프트 앞에 두면 컨텍스트 길이에 취약, 초안 품질 저하
Generate-then-Refine	먼저 제약 없이 최선의 초안을 생성 → 이후 짧고 집중된 교정 프롬프트로 수정

이 설계는 SLM 모델이 복잡한 조건부 생성을 직접 수행하는 대신, 두 번의 짧은 LLM 호출로 품질을 단계적으로 높입니다. 각 refine 함수는 `if not decision_guide: return draft`로 가드되어, KB에 해당 레벨의 가이드가 없을 경우 불필요한 LLM 호출을 방지합니다.

7. 데이터 스키마 (Data Schemas)

7.1 odyseuss_db.jsonl 레코드 스키마

```
{
  "task_id":      "string",
  "task":         "string",
  "true_answer":  "string",
  "agent_planning": "string | null",
  "task_analysis": {
    "knowledge":  "string",
    "plan":       ["step1", "step2", "..."],
    "task_type":  ["computation", "analysis", "..."],
    "domain":     ["mathematics", "science", "..."]
  },
  "plan_subtask_action": [
    {
      "step": "string",
      "subtasks": [
        {
          "subtask":      "string",
          "subtask_action": "string"
        }
      ]
    }
  ],
  "decision_augmentation": {
    "final_reference": {
      "knowledge": "string",
      "plan":      ["step1", "..."],
      "instance":  "string"
    },
    "signals_summary": [
      {
        "level":      "knowledge | plan | instance",
        "failures":   ["string"],
        "causes":     ["string"],
        "corrections": ["string"]
      }
    ]
  }
}
```

7.2 검색 결과 딕셔너리 스키마 (Retrieval Result Dict)

`AKB_Manager.hybrid_search()` 반환 리스트의 각 항목:

```
{
  "task_id":      str,
  "total_score":  float,      # hybrid / type_domain ■■
  "task":         str,
```

```

"true_answer":      str,
"agent_planning":   str | None,
"agent_experience":  str | None,
"task_analysis": {
    "knowledge": str | None,
    "plan":      list[str] | None,
    "task_type": list[str],
    "domain":    list[str],
} | None,
"plan_subtask_action": list | None,
"instance":            str | None,
"decision_guide": [
    {
        "level":      str,      # "knowledge" | "plan" | "instance"
        "failures":   list[str],
        "causes":     list[str],
        "corrections": list[str],
    }
] | None,
}

```

7.3 proposal_planning() 반환 딕셔너리 스키마

```

{
    "plan": str,
    # ■■ (plan_mode == None ■■ "plan"):
    # "Knowledge:\n{knowledge_text}\n\nPlan:\n{plan_str}\n\nInstance:\n{instance_str}"
    # subtask ■■:
    # "Knowledge:... \n\nPlan:... \n\nInstance:... \n\nSubtasks:\n{subtask_plan}"

    "knowledge":      str,      # Stage 1b ■■ (refined knowledge)
    "plan_steps":     str,      # Stage 2b ■■ (refined plan bullet list)
    "instance":       str,      # Stage 3b ■■ (refined instance)
    "retrieval_results": list,   # KB ■■ ■■ ■■
    "examples": {
        "plan_subtask":      str,      # build_plan_subtask_examples() ■■
        "plan_subtask_action": str,    # build_plan_subtask_action_examples() ■■
    },
}

```

부록 A. 태스크 분류 프롬프트 (Task Classification Prompt)

`retrieval_option == "type_and_domain"` 일 때 사용되는 분류 프롬프트입니다.

```

task_classification_prompt: |
    Given the task below, select all applicable task types and domains from the predefined lists.
    Choose only values that appear in the provided lists. You may select multiple values.

    TASK_TYPES (choose one or more):
    {{task_types}}

    DOMAINS (choose one or more):
    {{domains}}

    Return STRICT JSON (no extra keys, no prose outside the JSON):
    {
        "task_type_normalized": ["<type1>", ...],
        "domain_normalized": ["<domain1>", ...]
    }

    TASK:
    {{task}}

```

사전 정의된 상수 목록:

- TASK_TYPE_CONSTANTS: algebra, algorithm, analysis, approximation, calculation, computation, decision_support, engineering, geometry, simulations
 - DOMAIN_CONSTANTS: arts_humanities, business, computer_science, engineering, law, mathematics, medicine, science, social_sciences, technology
-

부록 B. 모듈 파일 경로 참조

역할	파일 경로
전체 실행 진입점	`examples/open_deep_research/run_gaia.py`
KB 인덱싱·검색	`examples/open_deep_research/agent_kb/agent_kb_retrieval.py`
Proposal Planning 로직	`examples/open_deep_research/planner_kb/planner.py`
프롬프트 템플릿	`examples/open_deep_research/planner_kb/planner_prompts.yaml`
Planner 공개 API	`examples/open_deep_research/planner_kb/__init__.py`
최종 답변 추출	`examples/open_deep_research/scripts/reformulator.py`